

Since you need both the **questions** and the **table structures** to practice effectively, I have organized 30 coding challenges into logical sets. I've provided the schema for each set so you can visualize exactly how to write your JOINS and WHERE clauses.

Set 1: Employee & Department (Questions 1–10)

Table: Employees (emp_id, name, dept_id, salary, manager_id)

Table: Departments (dept_id, dept_name, location)

1. **Simple Select:** List all employees in the 'Sales' department.
 2. **Top Salary:** Find the top 3 highest-paid employees.
 3. **Null Check:** Find all employees who do not have a manager assigned (manager_id is NULL).
 4. **Inner Join:** List every employee name along with their department name.
 5. **Aggregates:** Calculate the total salary spent per department.
 6. **Having Clause:** Find departments that have more than 10 employees.
 7. **Self Join:** Show the name of each employee and the name of their manager in the same row.
 8. **Subquery:** Find employees who earn more than the average salary of the entire company.
 9. **String Match:** Find all employees whose names start with 'J'.
 10. **Case Logic:** Create a column 'Salary_Class' that labels users as 'High' if salary > 10k and 'Low' otherwise.
-

Set 2: Customers & Orders (Questions 11–20)

Table: Customers (cust_id, first_name, country)

Table: Orders (order_id, cust_id, order_date, total_amount)

11. **Left Join:** List all customers and the total number of orders they've placed (include those with 0 orders).
12. **Date Filtering:** Find all orders placed in the last 30 days.
13. **Distinct:** Find all unique countries where your customers are located.
14. **Join & Sum:** Find the total revenue generated by customers from 'USA'.
15. **Exists:** Find customers who have placed at least one order using the EXISTS keyword.
16. **In-Clause:** Find orders placed by customers in either 'UK', 'France', or 'Germany'.
17. **Date Extraction:** Find the month in 2023 that had the highest total sales.
18. **Average:** Find the average order value (AOV) for customer ID #5.
19. **Between:** List orders with a total_amount between \$100 and \$500.
20. **Multi-Join:** If a Products table existed, how would you find the most popular product? (Describe the Join).

Set 3: Products & Inventory (Questions 21–30)

Table: Products (prod_id, prod_name, category, price, stock_qty)

21. **Update:** Write a query to increase the price of all 'Electronics' by 10%.
22. **Window Function (Rank):** Rank products by price within each category.
23. **Running Total:** Calculate the cumulative stock quantity as you go down the prod_id list.
24. **Duplicates:** Identify any prod_name that appears more than once in the table.
25. **Delete:** Remove products that have 0 stock and were created more than a year ago.
26. **Coalesce:** If prod_name is NULL, display 'Unnamed Product'.
27. **Math:** Find the total potential value of the inventory (price * stock_qty).
28. **Row Number:** Select only the 2nd most expensive product in the 'Furniture' category.
29. **Union:** Combine a list of prod_name from this table with a list of service_names from a Services table.
30. **CTE:** Use a Common Table Expression to find categories where the average price is above \$50.

Quick Syntax Cheat Sheet

If you are practicing these, remember the standard order of a SQL query:

1. **SELECT** (Choose columns)
2. **FROM** (Choose tables)
3. **JOIN** (Combine tables)
4. **WHERE** (Filter rows)
5. **GROUP BY** (Aggregate data)
6. **HAVING** (Filter groups)
7. **ORDER BY** (Sort results)
8. **LIMIT** (Restrict output)

Which of these would you like me to write the solution code for first?